2. There is definitely a pattern in the scatter plot of X_0 samples dependent on where the point ends up in the checkerboard. The tiles which end up in the middle are samples close to the mean of p_0. The mapping from X_0 also have the same positions relative to each other as in the checkerboard pattern. Pretty cool!

Question 3.3.5 (A/B)

A/B Assignment

In the basic assignment you saw that the baseline architecture with logit-normal sampling of the time variable during training more-or-less solved the Checkerboard pattern with "grid_sz=4". But one weakness of flow matching is its slowness to learn high-resolution patterns. In the A/B assignment you will explore this issue by increasing the high-resolution of the Checkerboard pattern by setting "grid_sz=8". Re-train the baseline network for this data-distribution with 10000 update steps. Notice you get a huge drop in performance. Your challenge is to train a network that for 2,000 points generated from random samples of p_init using your flow-matched trained network achieves a precision score of $\geq .86$ while keeping the recall high. You are limited to 50,000 update steps during training. You are, of course, free to use whichever approaches/improvements you like but we have suggested some useful tips:

1. Use Fourier feature mappings to allow neural networks to more effectively learn high-frequency functions from low-dimensional inputs. For example the approach used in **One Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains**, Tancik et al, NeurIPs 2020 (https://arxiv.org/abs/2006.10739). The core idea is to map

   $$x \in \mathbb{R}^d \text{ to } [\sin(Bx), \cos(Bx)]$$

   where

   $$B \text{ is } n_f \times d \text{ and each } B_{ij} \sim \mathcal{N}(0, \sigma^2).$$

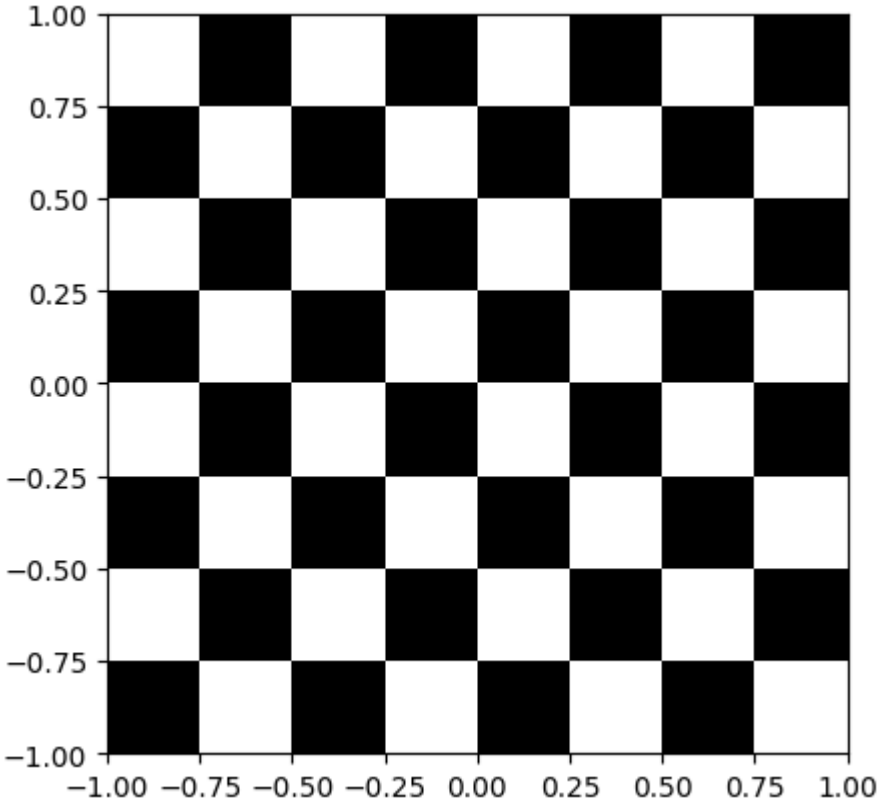
   This feature mapping is known as Random Fourier Features. In the literature there are multiple potential improvements to Random Fourier Features such as learning B. However, these improvements may require some more fiddling to get training to work well.
   To fit in with the provided code base you will have to make adjusts to MLP_fn w.r.t. the input dimensions of the defined MLP and how the t and x_t are concatenated and mapped before being entered into the network. FYI in my experiments I concatenated t and x_t and applied B to the resulting 3 dimensional vector. And this worked well. I did not explore other possible variations!
2. Make the architecture of the baseline network wider and perhaps shallower (to make training easier and quicker).

3. Definitely train for longer than the 10000 steps in the basic assignment
4. Better optimization of the hyper-parameters of the logit-normal may also help.

Let us see what the new grid looks like!

In [126…

```
# initialize class for the Checkerboard pattern
grid_sz = 8
p_data = a3_utils.CheckerboardSampleable(device, grid_size=grid_sz)

# display the checkerboard pattern
fig = plt.figure()
axes = fig.add_subplot(1, 1, 1)
p_data.plot_distribution(axes)
```



Let us start by training a baseline model that uses the same parameters, logit_normal sampling, and midpoint ODE solver as above but with 50000 update steps which is the maximum allowed update steps for this part. Increasing update steps will hopefully make the model fit to the new grid size reasonably well.

In [127…

```
# train with logit-normal sampling of time during training
logit_train_sampling = TimeSampling('logit_normal', logit_mean = 1.0, logit_scale = 1.0)
logit_training_params = {'n_batch': 1024, 'n_update': 50000, 'time_sampling': logit_train_sampling}

# change network parameters
network_params = {'L': 4, 'h': 64}
# run training
vf_fn_logit = run_training(network_params, logit_training_params)
```

```
Training vf network....
Training 50000 steps took 343.9191 seconds
```

In [128…

```python
# I modify some of the given functions a bit so that I only need to generate samples once
def get_samples(gen_samples, n_samples, n_steps):
    X_0 = torch.randn(n_samples, 2).to(device)
    X_t = gen_samples.generate(X_0, n_steps=n_steps)
    return X_0, X_t

def measure_performance3(X_t, p_data):
    precision, recall_norm, f_score, per_tile_distribution = p_data.evaluate_points_precision_entropy(X_t)
    print(f"Precision: {precision:.4f}")
    print(f"Normalized Recall: {recall_norm:.4f}")
    print(f"Harmonic Mean (F1-score): {f_score:.4f}")
```

Let us first draw 2000 samples to get an "official" record of the performance of the baseline model.

In [141…

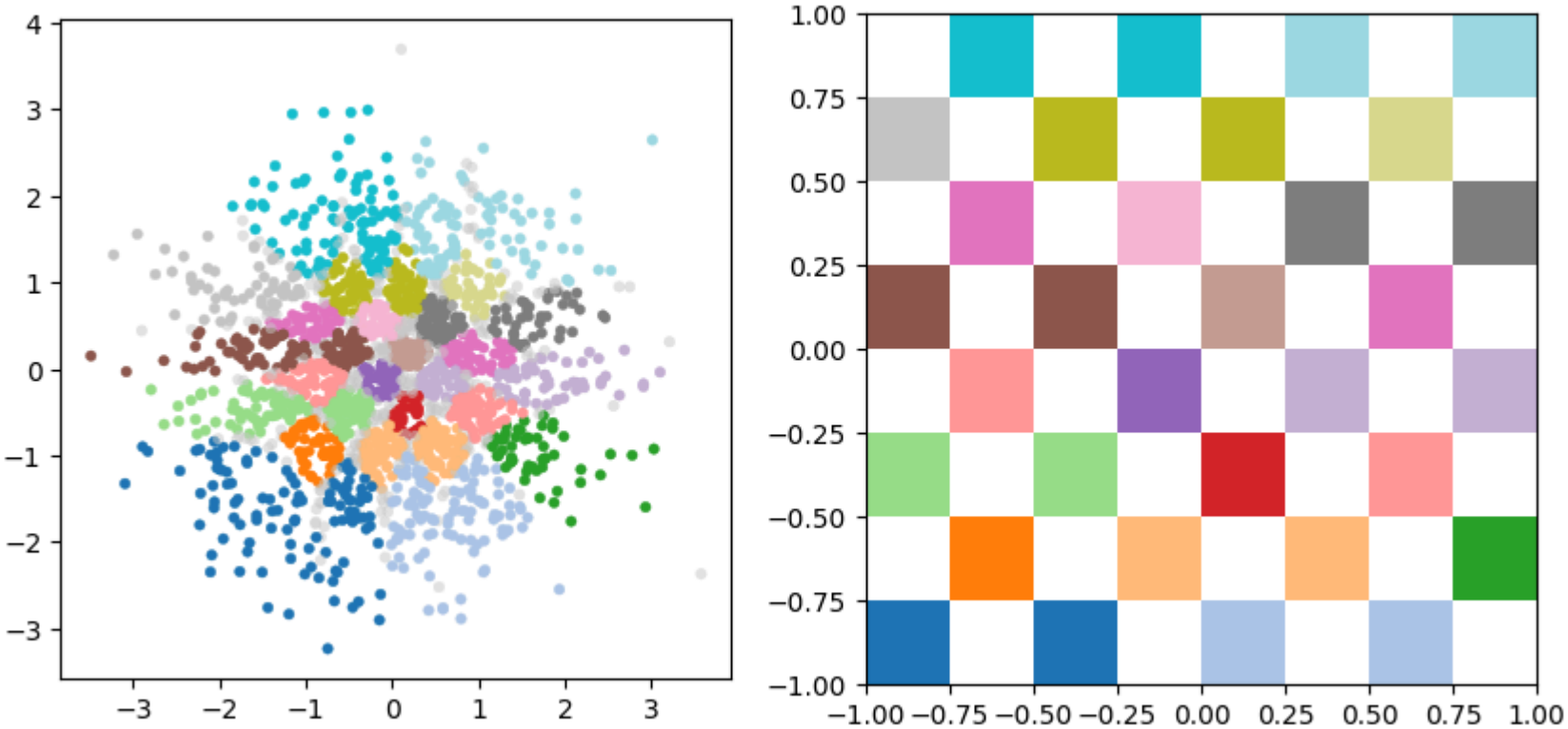
```python
# Generate samples and measure performance of the baseline
n_samples = 2000
n_steps = 30
gen_samples = GenerateSamples(update_step_midpoint_ode, vf_fn_logit)
X_0, X_t = get_samples(gen_samples, n_samples, n_steps)

print('Baseline performance')
measure_performance3(X_t, p_data)
```

```
Baseline performance
Precision: 0.8265
Normalized Recall: 0.9964
Harmonic Mean (F1-score): 0.9035
```

Let us print the results so that we can compare it to our final model later!

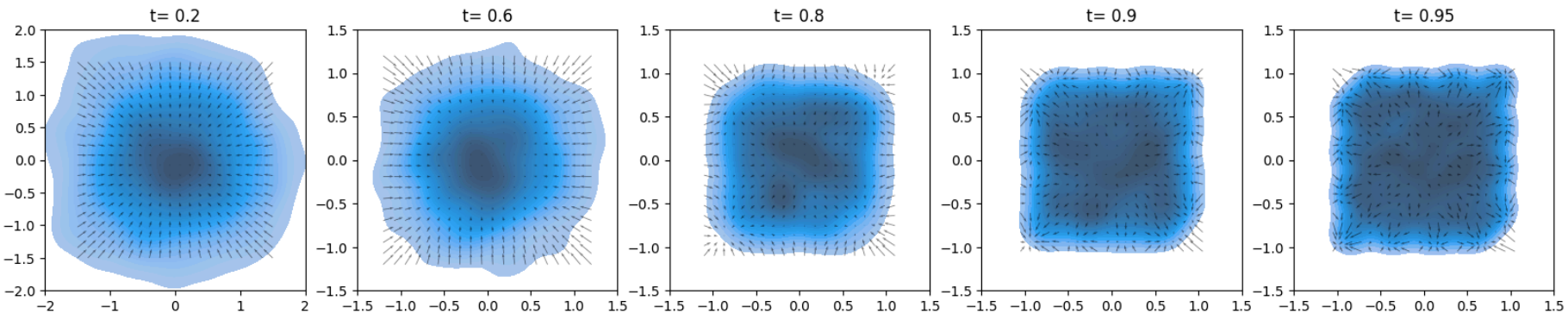
In [142…

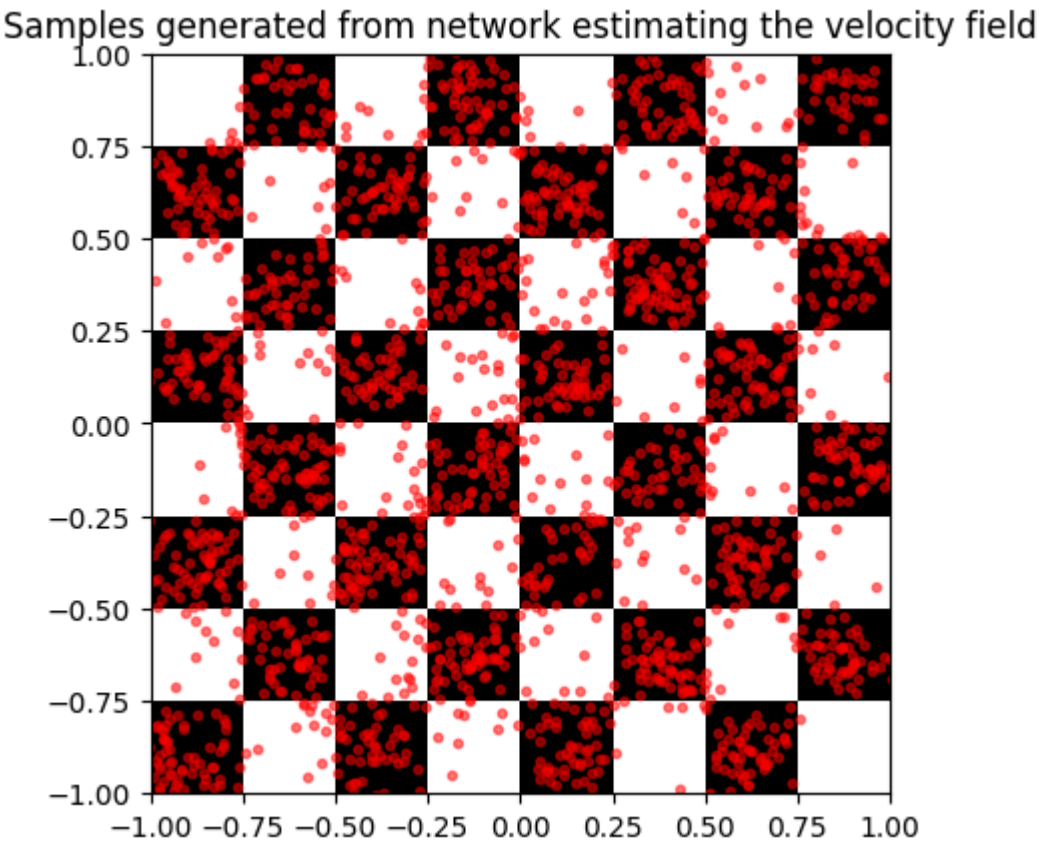
```python
def plot_X_0(gen_samples, p_data, X_0, X_t):
    labels = p_data.label_points(X_t)
    a3_utils.visualize_noise_to_which_tile(X_0, labels, p_data)

plot_X_0(midpoint_gen_samples, p_data, X_0, X_t)
```



In [143…

```
fig = plt.figure(figsize=(20, 5))
fig.clear()
a3_utils.visualize_learnt_vf(vf_fn_logit, ts, X_t, p_data, fig, device, num_samples=2000, num_bins=20)
```





Since it is cheap to draw samples from the baseline model we can draw more samples here. This is to confirm the capacity a bit more thorough, we do not want to stumble on some nice number by chance. When drawing 2000 samples the precision is extremely dependent on the samples, for example precision varies by multiples of +-0.01 for the baseline model if I draw 2000 samples two times. Therefore, I will have to be careful going forward and evaluate either 2000 samples multiple times or draw more samples when it is computationally achievable. I resort to the former since the precision metric we are trying to "optimize" is for 2000 samples. Therefore it might be useful to look at the mean and std for multiple rounds of drawing 2000 samples.

Let us repeatedly draw 2000 samples from the baseline model to see just how much the performance metrics can vary to see how reliable our current results are.

In [146…

```python
def measure_performance4(X_t, p_data):
    precision, recall_norm, f_score, per_tile_distribution = p_data.evaluate_points_precision_entropy(X_t)
    return precision, recall_norm, f_score
```

In [148…

```python
iterations = 20
n_samples = 2000
n_steps = 30

precision_array = np.zeros((iterations, 3))

for i in range(iterations):
    gen_samples = GenerateSamples(update_step_midpoint_ode, vf_fn_logit)
    X_0, X_t = get_samples(gen_samples, n_samples, n_steps)
    precision_array[i,0],precision_array[i,1],precision_array[i,2] = measure_performance4(X_t, p_data)
precision_mean = np.mean(precision_array, axis=0)
precision_std = np.std(precision_array, axis=0)

print('Multiple rounds of generated samples for the baseline model and corresponding performance')

print(f"Precision mean: {precision_mean[0]:.4f}")
print(f"Normalized Recall mean: {precision_mean[1]:.4f}")
print(f"Harmonic Mean (F1-score) mean: {precision_mean[2]:.4f}")

print(f"Precision std: {precision_std[0]:.4f}")
print(f"Normalized Recall std: {precision_std[1]:.4f}")
print(f"Harmonic Mean (F1-score) std: {precision_std[2]:.4f}")
```

```
Multiple rounds of generated samples for the baseline model and corresponding performance
Precision mean: 0.8295
Normalized Recall mean: 0.9961
Harmonic Mean (F1-score) mean: 0.9052
Precision std: 0.0081
Normalized Recall std: 0.0008
Harmonic Mean (F1-score) std: 0.0048
```

I now try a few experiments where I change the width and depth of the network and the scale and mean parameter of the logit normal sampling.

The experiments do not give better precision or F1 than the baseline model. I therefore continue by implementing the suggest Fourier feature mapping like described below (and in the instructions):

Use Fourier feature mappings to allow neural networks to more effectively learn high-frequency functions from low-dimensional inputs. For example the approach used in **One Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains**, Tancik et al, NeurIPs 2020 (https://arxiv.org/abs/2006.10739). The core idea is to map

$$\mathbf{x} \in \mathbb{R}^d \text{ to } [\sin(2\pi B\mathbf{x}), \cos(2\pi B\mathbf{x})]^T$$

where

$$B \text{ is } n_f \times d \text{ and each } B_{ij} \sim \mathcal{N}(0, \sigma^2).$$

This feature mapping is known as Random Fourier Features. In the literature there are multiple potential improvements to Random Fourier Features such as learning B. However, these improvements may require some more fiddling to get training to work well. To fit in with the provided code base you will have to make adjusts to MLP_fn w.r.t. the input dimensions of the defined MLP and how the t and x_t are concatenated and mapped before being entered into the network. FYI in my experiments I concatenated t and x_t and applied B to the resulting 3 dimensional vector. And this worked well. I did not explore other possible variations!

Let us modify the MLP_fn accordingly:

In [149…

```
# MLP function class
class MLP_fn(torch.nn.Module):

    def __init__(self, network_params):

        super().__init__()
        dim = 2

        in_dim = dim + 1

        h = network_params['h']
        L = network_params['L']
        num_fourier_features = network_params['num_fourier_features']
        sigma = network_params['sigma']
```

```python
        # Create the random matrix B for Fourier Features
        # We set the input to 3 dimensions like described above (t, x_t), B then has dimensions num_fourier_features x 3
        self.B = torch.nn.Parameter(torch.randn(num_fourier_features, in_dim) * sigma, requires_grad=False)

        # The new input dimension for the first layer is 2 * num_fourier_features
        fourier_feature_dim = 2 * num_fourier_features

        mlp = []
        mlp.append(torch.nn.Linear(fourier_feature_dim, h))
        mlp.append(torch.nn.SiLU())

        for idx in range(L):
            mlp.append(torch.nn.Linear(h, h))
            mlp.append(torch.nn.SiLU())
        mlp.append(torch.nn.Linear(h, dim))

        self.net = torch.nn.Sequential(*mlp)


    def forward(self, t: Tensor, x_t: Tensor) -> Tensor:

        # Concatenate t and x_t
        x_t_cat = torch.cat((t, x_t), -1)

        # Apply the Fourier Feature mapping
        # x_t_cat @ self.B.T is equivalent to Bx but with the correct expected dimensions
        # for the rest of the network (batch_size x num_fourier_features).
        # Also multiply with 2 pi like in the original paper.
        fourier_features = (2. * torch.pi * x_t_cat @ self.B.T)

        # Mapping [sin(Bx), cos(Bx)]
        fourier_features_sin = torch.sin(fourier_features)
        fourier_features_cos = torch.cos(fourier_features)

        # Concatenate the sine and cosine features
        fourier_feature_vector = torch.cat((fourier_features_sin, fourier_features_cos), -1)

        return self.net(fourier_feature_vector)
```

Let us now train with Fourier features. We probably do not need 50000 update steps so let us try 10000 like we used in the basic assignment. We start by selecting some reasonable starting points for sigma and num_fourier_features.

In [150…

```
# train with logit-normal sampling of time during training
logit_train_sampling = TimeSampling('logit_normal', logit_mean = 1.0, logit_scale = 1.0)
logit_training_params = {'n_batch': 1024, 'n_update': 10000, 'time_sampling': logit_train_sampling}

# change network parameters
network_params = {'L': 4, 'h': 64, 'num_fourier_features' : 512, 'sigma': 1}
# run training
vf_fn_logit = run_training(network_params, logit_training_params)
```

```
Training vf network....
Training 10000 steps took 173.0776 seconds
```

In [151…

```
# Generate samples and measure performance
n_samples = 2000
n_steps = 30
gen_samples = GenerateSamples(update_step_midpoint_ode, vf_fn_logit)
X_0, X_t = get_samples(gen_samples, n_samples, n_steps)

measure_performance3(X_t, p_data)
```

```
Precision: 0.9120
Normalized Recall: 0.9971
Harmonic Mean (F1-score): 0.9527
```

That is a great improvement, we are now consistently getting precision > 0.89 (mean around 0.91) while keeping normalized recall > 0.99.

Let us calculate the mean and variance of the performance metrics when drawing 2000 samples 20 times:

In [153…

```
iterations = 20
n_samples = 2000
n_steps = 30

precision_array = np.zeros((iterations, 3))

for i in range(iterations):
    gen_samples = GenerateSamples(update_step_midpoint_ode, vf_fn_logit)
```

```
    X_0, X_t = get_samples(gen_samples, n_samples, n_steps)
    precision_array[i,0],precision_array[i,1],precision_array[i,2] = measure_performance4(X_t, p_data)
precision_mean = np.mean(precision_array, axis=0)
precision_std = np.std(precision_array, axis=0)

print('Multiple rounds of generated samples for the improved model with Fourier features and corresponding performance')

print(f"Precision mean: {precision_mean[0]:.4f}")
print(f"Normalized Recall mean: {precision_mean[1]:.4f}")
print(f"Harmonic Mean (F1-score) mean: {precision_mean[2]:.4f}")

print(f"Precision std: {precision_std[0]:.4f}")
print(f"Normalized Recall std: {precision_std[1]:.4f}")
print(f"Harmonic Mean (F1-score) std: {precision_std[2]:.4f}")
```

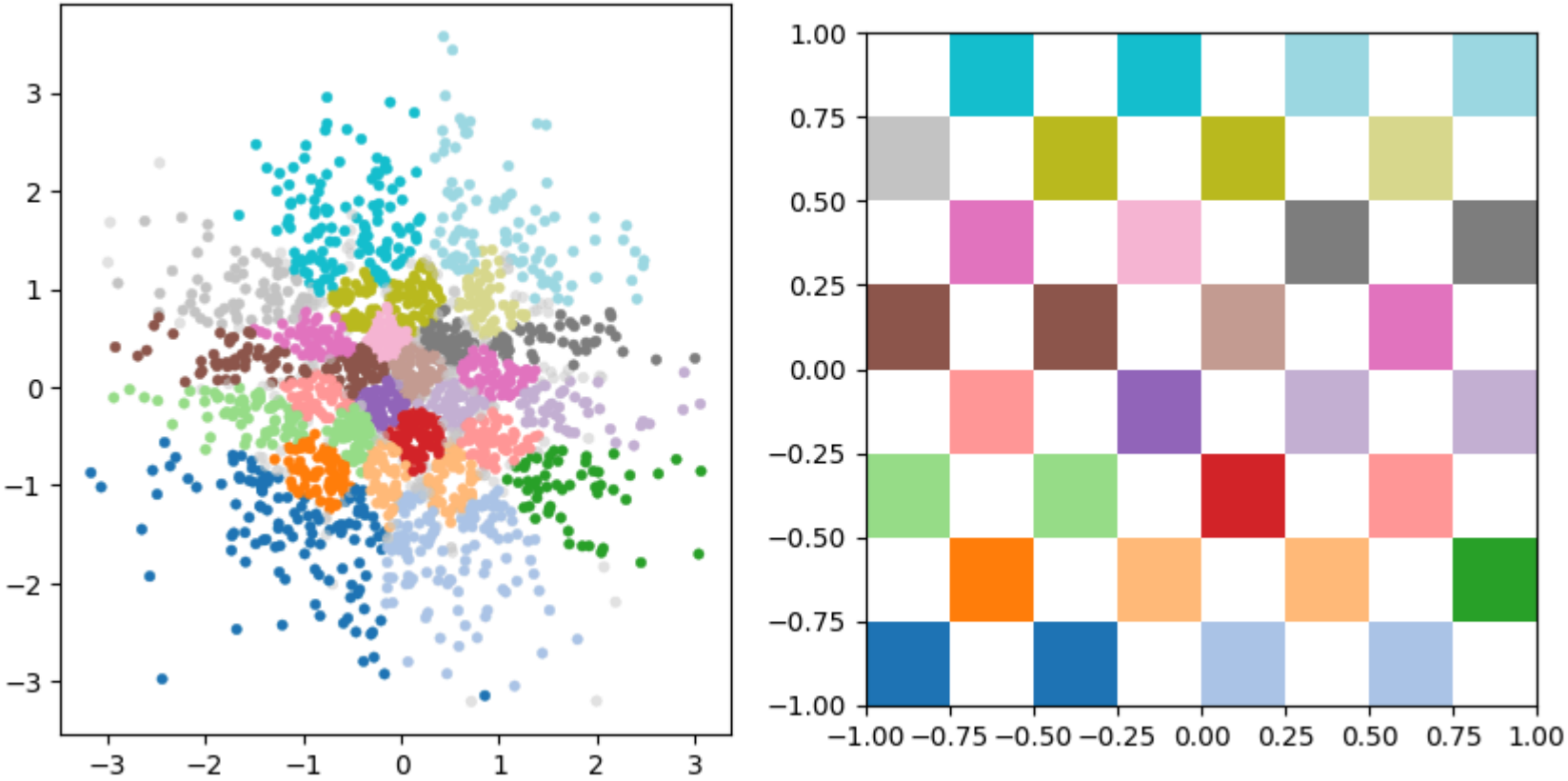
```
Multiple rounds of generated samples for the improved model with Fourier features and corresponding performance
Precision mean: 0.9103
Normalized Recall mean: 0.9956
Harmonic Mean (F1-score) mean: 0.9510
Precision std: 0.0046
Normalized Recall std: 0.0006
Harmonic Mean (F1-score) std: 0.0025
```

We achieved a precision well above the threshold, nice! It also beats the baseline by around 0.08 in terms of precision and 0.04 in terms of F1-score. Let us look at some plots and summarize our final performance metrics.

Cool! Let us plot and compare to the baseline model.

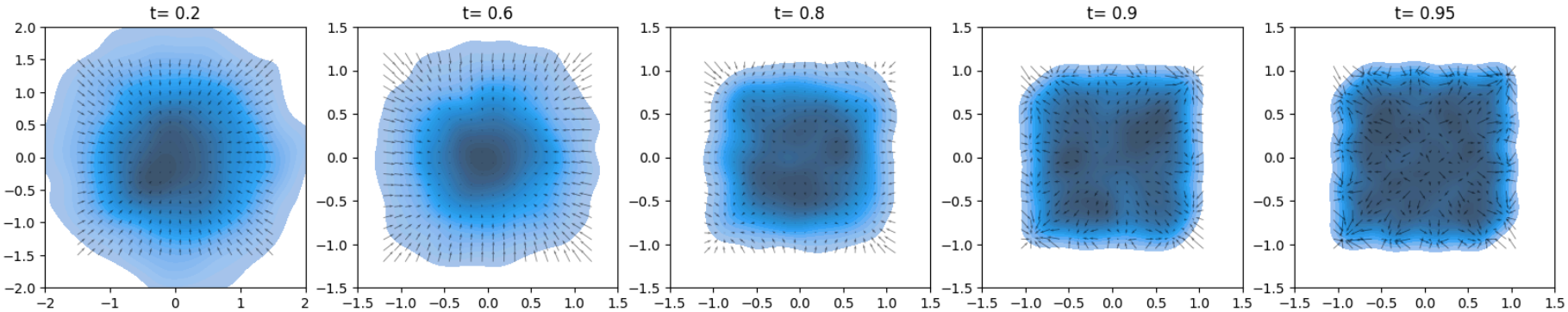
In [154…

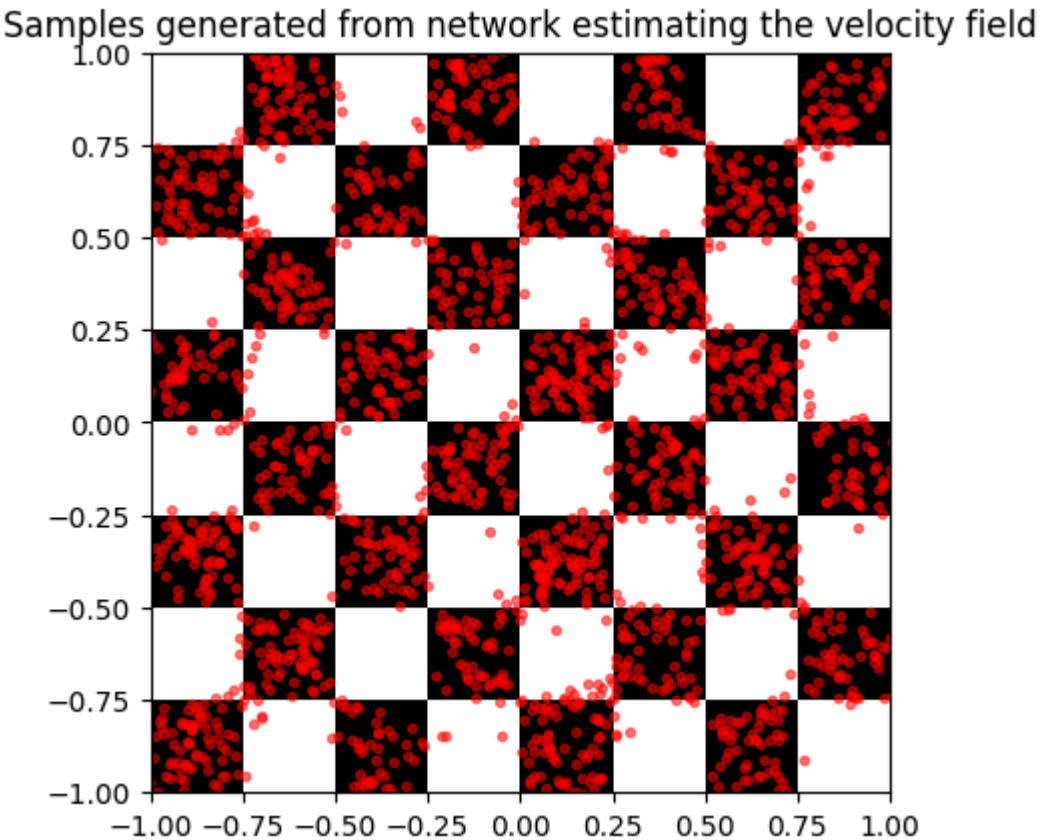
```
plot_X_0(midpoint_gen_samples, p_data, X_0, X_t)
```



In [155…

```
# Visualize the results
fig = plt.figure(figsize=(20, 5))
fig.clear()
a3_utils.visualize_learnt_vf(vf_fn_logit, ts, X_t, p_data, fig, device, num_samples=2000, num_bins=20)
```





Metrics of the final precision model:

Precision: 0.9103 +-0.0046
Normalized Recall: 0.9956 +-0.0006
Harmonic Mean (F1-score): 0.9510 +-0.0025

Question 3.3.6 (E)

Please declare if you used any generative AI tools for solving this notebook, and if so which tools and roughly which parts you used them for. If you did not any generative AI tools, just answer "None". If you did use generative AI tools, also reflect in a sentence or two on how well it worked and what you learnt.

None

Acknowledgements: Please note the structure of the pdf sampling and FM training of the neural network code is copied and/or adapted from https://github.com/eje24/iap-diffusion-labs/blob/main/solutions/lab_two_complete.ipynb and https://github.com/facebookresearch/flow_matching